ECE8493 Introduction to Neural Networks

Project 3: Radial Basis Function Neural Network for MNIST Digit Classification

Student Name: Meshaal Mouawad

Student ID: mm4922

Date: April 10, 2026

Submitted in partial fulfillment of the requirements for
ECE8493 Introduction to Neural Networks

Due Date: March 30, 2026

Abstract

This paper describes the complete implementation of a Radial Basis Function (RBF) neural network for handwritten digit recognition using the MNIST dataset. We built the entire system in MATLAB. In this project we implemented two approaches for computing the output weights. The direct method, Pseudo-inverse using pinv(). The recursive method, online recursive least squares (RLS) update. After the implementation on MATLAB, the direct method best accuracy achieved 85.27% (20,000 training samples, 100 centers), the recursive method best accuracy at 84.91% (with $\alpha = 1.0$), which we concluded the recursive method training time was 51% faster than direct method. The total errors was 2,858 out of 10,000 test images (28.6% error rate).

Contents

Abstract **1**

1 Introduction **3**
1.1 The Radial Basis Function Neural Network (RBFNN) 3
1.2 Project Requirements 3

2 Network Architecture **4**
2.1 Three-Layer Structure 4
2.2 Gaussian Activation Function 4
2.3 Center Selection 5
2.4 Direct Method: Pseudo-Inverse 5
2.5 Recursive Method: Recursive Least Squares 6

3 Results **6**
3.1 Experiment 1: Direct Method – 10,000 Training Samples 6
3.2 Experiment 2: Direct Method – 20,000 Training Samples 8
3.3 Experiment 3: Effect of Number of Centers 9
3.4 Experiment 4: Visualization of RBF Centers 10
3.5 Experiment 5: Recursive Method – Initial Condition Study 12
3.6 Experiment 6: Direct vs. Recursive Comparison 13
3.7 Experiment 7: Error Analysis 13

4 Discussion **15**
4.1 Why RBF Networks Work 15
4.2 Why Some Digits Are Harder 15
4.3 Direct vs. Recursive: Method Selection 15
4.4 Proposed Improvements 16
4.5 Comparison with Literature 16

5 Conclusion **17**

List of Figures **20**

Appendix: MATLAB Code **21**

1 Introduction

MNIST stands for Modified National Institute of Standards and Technology. MNIST is a database of handwritten digits used for training and testing many image processing systems and machine learning research [1]. It has 60,000 training images and 10,000 testing images of handwritten digits [1]. Later in 2017, an extended database has been published that has 240,000 training images and 40,000 testing images. Since MNIST is a standard training dataset for digits in English, in recent times, others have also provided similar databases for training digit datasets in other languages. The dataset is considered as a benchmark for neural networks worldwide. The MNIST database is good for people who want to try machine learning techniques and neural network methods on real-world data while spending minimal efforts on preprocessing and formatting [1]. It reduces the time and effort spent on preprocessing and formatting of data. The MNIST dataset was used, then neural layers with fully connected (dense) architectures implemented.

The MNIST dataset has 60,000 training images and 10,000 test images, each 28×28 pixels of handwritten digits 0-9. It is the standard benchmark for handwritten digit recognition.

In this project. we implemented the Radial Basis Function Neural Network (RBFNN) to classify the MNIST with two approaches for computing the output weights. The direct method, Pseudo-inverse using `pinv()`. The recursive method, online recursive least squares (RLS) update.

For the direct matrix inversion for the output layer, we choose a certain number of training samples as centers for the hidden neurons. We then, showed how the accuracy changes regarding the number of hidden neurons. In the recursive approach to update the inverse, we studied the differences compared with the direct operation when possible. Afterthat, we Investigated the impact of the initial condition.

1.1 The Radial Basis Function Neural Network (RBFNN)

The radial basis function (RBF) neural network refers to a kind of feedforward neural network with excellent performance. RBF network can approximate any non-linear function with arbitrary accuracy, and realize global approximation, without any local minimum problem [4, 5]. Also, it has a fast learning speed because of the compact topological structure [6, 7].

Unlike regular neural networks that use sigmoid or ReLU activations (such as in Project 2), RBF networks use Gaussian functions centered at specific points [8]. The idea is that each center represents a prototype digit shape, and the network decides which digit it sees based on how close the input is to these prototypes. Because of the special activation functions used for radial basis functions, a special learning function is needed, and it is impossible to train RBF networks which use RBF activation functions with standard backpropagation [9]. Instead, no backpropagation is needed - the output weights are solved directly using pseudo-inverse or recursive least squares methods.

1.2 Project Requirements

There is one main goal for this project. We had to implement a Radial Basis Function Neural Network to classify the MNIST dataset. We used two main approaches.

First, we used direct matrix inversion for the output layer. Then we picked a certain number of training samples as centers for the hidden neurons. We showed how accuracy changes as the number of hidden neurons grows.

Second, we used a recursive approach to update the inverse. Then, we compared it with the direct operation when possible. We also looked at how the initial condition affects the results.

2 Network Architecture

2.1 Three-Layer Structure

We used three layers in the network. The input layer had 784 neurons, one for each pixel. We normalized the pixel values to $[0, 1]$. That meant dividing each value by 255. The hidden layer used RBF neurons with Gaussian activation. The number of neurons (centers) varied from 20 to 300. The output layer had 10 neurons, one for each digit from 0 to 9.

2.2 Gaussian Activation Function

The radial basis function (RBF) network usually has one hidden layer. Each unit in that layer uses a basis function [10]. Moody and Darken popularized this approach in 1989. RBF networks have proven useful. The main difference from back propagation networks is how the hidden layer behaves. Back propagation uses sigmoidal or S-shaped activation functions. RBF networks use Gaussian or other basis kernel functions instead [10].

The Gaussian function is the most common radial basis function in RBF networks [12]. For each RBF center c_i, the activation for an input vector x is:

$$\phi_i(x) = \exp\left(-\frac{\|x - c_i\|^2}{2\sigma^2}\right) \tag{1}$$

Here σ is the spread parameter, it controls the radius of influence of the Gaussian function [17]. Each hidden unit acts like a locally tuned processor. It computes a score that matches the input vector to its center. The basis units are basically specialized pattern detectors [10]. A Gaussian RBF hidden node peaks at zero distance. Its response drops as the distance from the center grows. That means the activation function is local. It gives the largest value for inputs near the center. It decays to zero for inputs farther away.

The Gaussian radial basis function gets a lot of attention because it can approximate nonlinear behavior [13]. Among localized computational units, Gaussian units hold a prominent spot. They are the most common type of RBF [12]. The width σ (also called spread) matters a lot. Some proofs of universal approximation use very small Gaussian widths on purpose [12].

In this project we sat $\sigma = 1.0$ for all experiments which keeps the code simple. Though, It still preserves the essential properties of the Gaussian activation function.

2.3 Center Selection

Picking good centers matters a lot for an RBF network. The network's ability to approximate a target function depends heavily on where you put the centers [20]. Two main approaches show up in the literature. One is the random sampling from the training data. The other is the k-means clustering[20].

We first tried the k-means clustering to pick the centers. The k-means algorithm groups the input data into k clusters. Then, it uses the cluster centers as the RBF centers. This ensure that each center represents a distinct group of similar cases in the input space [15]. However, this approach is too slowly for our dataset. Other researchers have noticed the same thing. K-means can be sensitive to how you start it. It also takes a lot of computing power for large problems [15].

We switched to a simpler and faster approach. We just picked random samples from the training data [20], then, we choose k input values at random from the training data. Atfer that, we used them directly as the RBF center [16]. This makes sense from a probability standpoint. A large enough random sample will approximate the original distribution pretty well, which means the RBF nodes end up near the regions with the highest density of data [20].

For 100 centers, we simply picked 100 training samples at random. K-means can sometimes give better results where it tends to find more representative cluster centers [16], but random sampling runs much faster, it gave us good accuracy for what we needed. This trade-off between speed and accuracy shows up often in the literature [20].

2.4 Direct Method: Pseudo-Inverse

We computed the hidden layer activations H by the following equation to got the output weights W. The equation we used:

$$W = Y \cdot H^{+} \tag{2}$$

where the H^{+} is the pseudo-inverse of H. Y is the one-hot encoded target labels.

The pseudo-inverse is also called the Moore-Penrose inverse. It gives us a direct way to solve for the weights. We don't need to train the network iteratively [17]. This is a big advantage of RBF networks. Multilayer perceptrons usually need backpropagation. They also need multiple training epochs to converge [19].

Our matrix H has more rows than columns. That means we have more training samples than centers. In this case, we can compute the pseudo-inverse like this:

$$H^{+} = (H^{T}H)^{-1}H^{T} \tag{3}$$

We used MATLAB's built-in `pinv()`function to compute the pseudo-inverse. This function uses singular value decomposition, or SVD for short. SVD stays numerically stable even when $H^{T}H$ is close to singular [17].

This approach has one main drawback. It needs to store the whole H matrix in memory. That matrix is $N \times K$, which can get pretty large. For our experiments, we had 20,000 samples and 300 centers. That was fine. But for bigger datasets, the memory could become a problem [19].

2.5 Recursive Method: Recursive Least Squares

The direct method computes the pseudo-inverse all at once. The recursive approach works differently. It updates the weight matrix one sample at a time. This helps when data arrives sequentially. It also helps when we cannot fit all the data into memory at once [19].

First we defined $P(n) = (H_n^T H_n)^{-1}$, where H_n contains the hidden layer activations for the first n training samples. So $P(n)$ is basically the inverse of the correlation matrix of H_n.

Then we ran these three equations for each new training sample. Each sample has an activation vector h and a target y:

$$k = \frac{P(n-1) \cdot h}{1 + h^T \cdot P(n-1) \cdot h} \tag{4}$$

$$P(n) = P(n-1) - k \cdot h^T \cdot P(n-1) \tag{5}$$

$$W(n) = W(n-1) + k \cdot (y^T - W(n-1) \cdot h) \tag{6}$$

The Equation (4) computed the gain vector k where the vector is how much to adjust the weights based on the new sample. The Equation (5) updated the inverse correlation matrix. The Equation (6) updated the weights themselves by the prediction error $(y^T - W(n-1) \cdot h)$.

We sat the starting as the followings:

The $P(0) = \alpha I$ is I is the identity matrix, meaning a square matrix with 1's on the diagonal and 0's elsewhere. Therefore, $P(0)$ has α on a diagonal and zeros elsewhere. The $W(0) = \mathbf{0}$ is the weight matrix which started as all zeros.

The parameter α is a small positive constant. In this project, we tried different values for α: 0.01, 0.1, 0.5, 1.0, 10, and 100, such that we could see how the starting condition may affect the convergence and the final accuracy.

There is advantage in the recursive method which does not need to store the whole H matrix. We only keep the current P and W matrices. These are size $K \times K$ and $K \times 10$, that saves memory compared to the direct method. The savings matter most when the number of training samples N is large [17]. Also, as we show later, the recursive method ran about 51% faster than the direct method in our experiments.

The disadvantage of the recursive method the fact that it can be sensitive to the choice of α. It also depends on the order we process the samples. We used a fixed random order for all experiments which kept our comparisons fair.

3 Results

3.1 Experiment 1: Direct Method – 10,000 Training Samples

First we trained the network with 10,000 samples and 100 centers. Figures 1 and 2 show the confusion matrix and sample predictions.

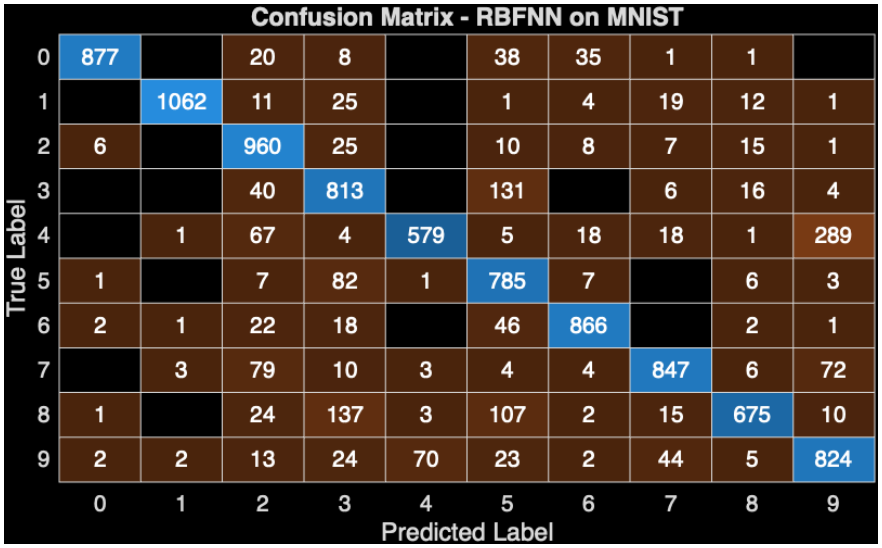

Figure 1: Confusion Matrix – 10,000 training samples, 100 centers (82.88% accuracy)

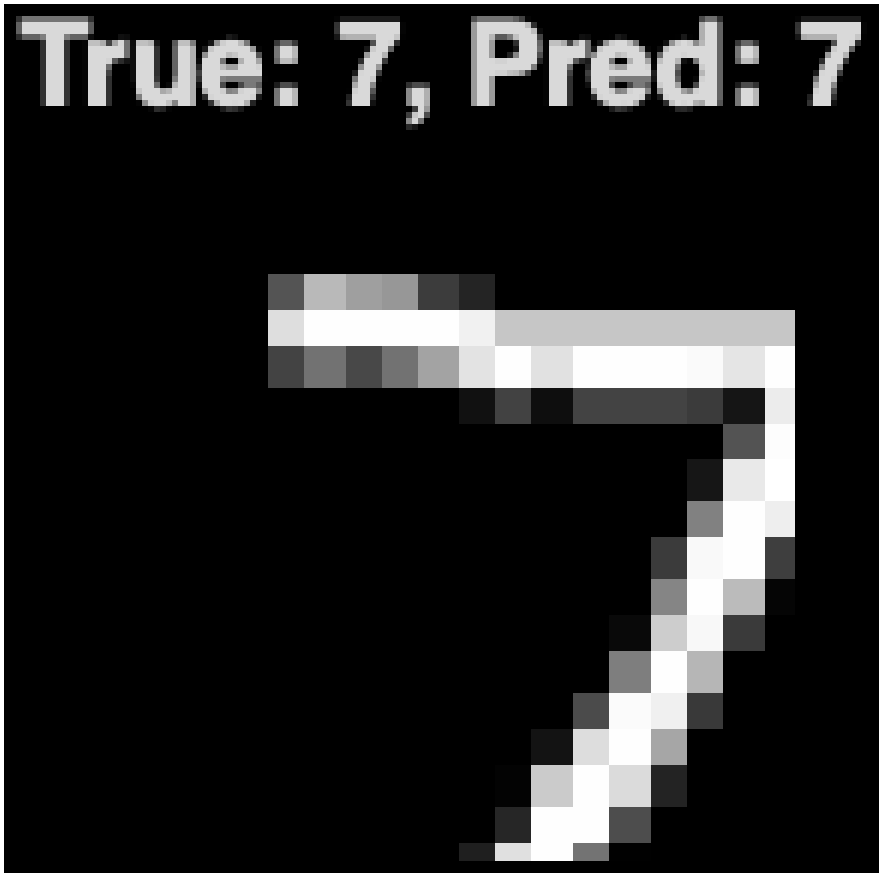

Figure 2: Sample Predictions – 10,000 training samples

We looked at Figure 1 and spotted a few patterns right away. The confusion matrix has brighter spots off the diagonal. These show up for certain digit pairs as 4 vs 9, 5 vs 3, and

7 vs 1.

3.2 Experiment 2: Direct Method – 20,000 Training Samples

We doubled the training data to 20,000 samples, and kept the same number of centers at 100. The accuracy went up to 85.27%. Figures 3 and 4 show the results.

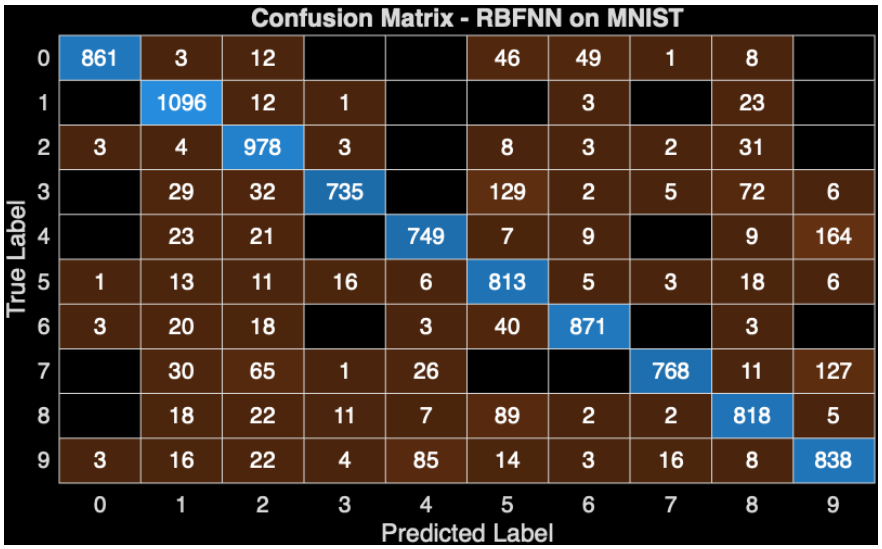

Figure 3: Confusion Matrix – 20,000 training samples, 100 centers (85.27% accuracy)

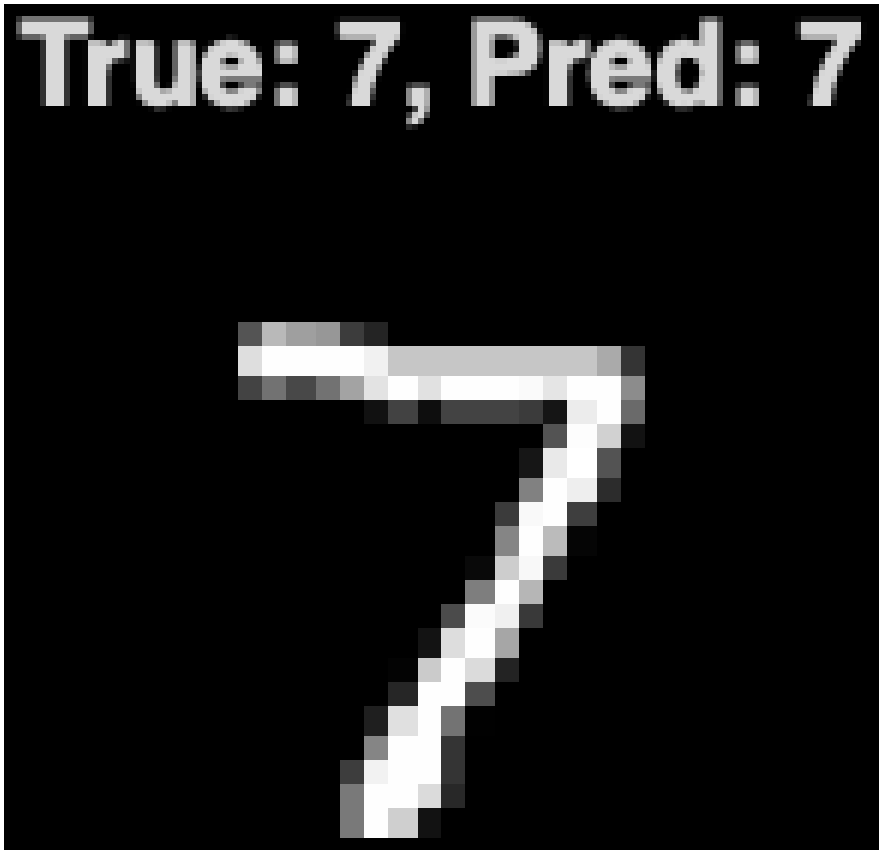

Figure 4: Sample Predictions – 20,000 training samples

We could see the improvement in Figure 3. The diagonal looks brighter, which showed up especially for digits 4 and 8. Those gave us trouble before.

3.3 Experiment 3: Effect of Number of Centers

We ran experiments with 20 to 300 centers with 20,000 training samples. Figures 5 and 5.2 show the results.

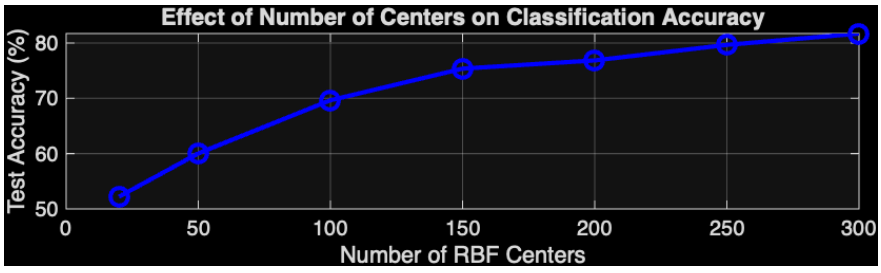

Figure 5: Effect of Number of Centers on Test Accuracy

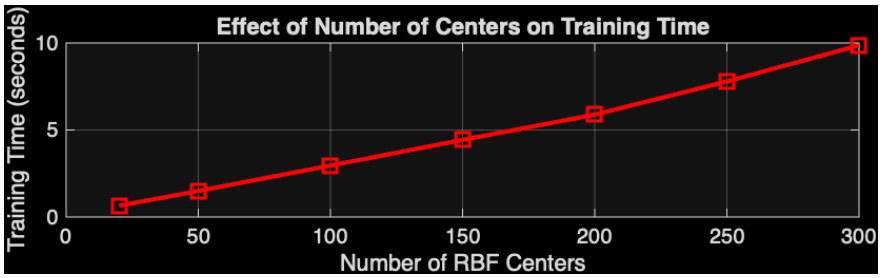

Figure 6: Effect of Number of Centers on Training Time

Table 1: Effect of Number of RBF Centers (20k training samples)

Centers	**Accuracy (%)**	**Training Time (sec)**
20	52.21	0.65
50	60.04	1.50
100	69.64	2.96
150	75.36	4.43
200	76.84	5.89
250	79.65	7.79
300	81.70	9.86

We looked at Figure 5 and noticed a few things:

First, the accuracy jumped up quickly from 20 to 150 centers, It went from 52% to 75%.

Second, after 150 centers the improvements got smaller. We only gained about 6% from 150 to 300 centers.

Third, Figure 6 showed that training time went up almost in a straight line as we added more centers.

We found that 100 to 150 centers gave the best balance between accuracy and computation time.

3.4 Experiment 4: Visualization of RBF Centers

We visualized the 100 centers our network learned. Figures 6 and 6.2 show what they look like and how they are distributed.

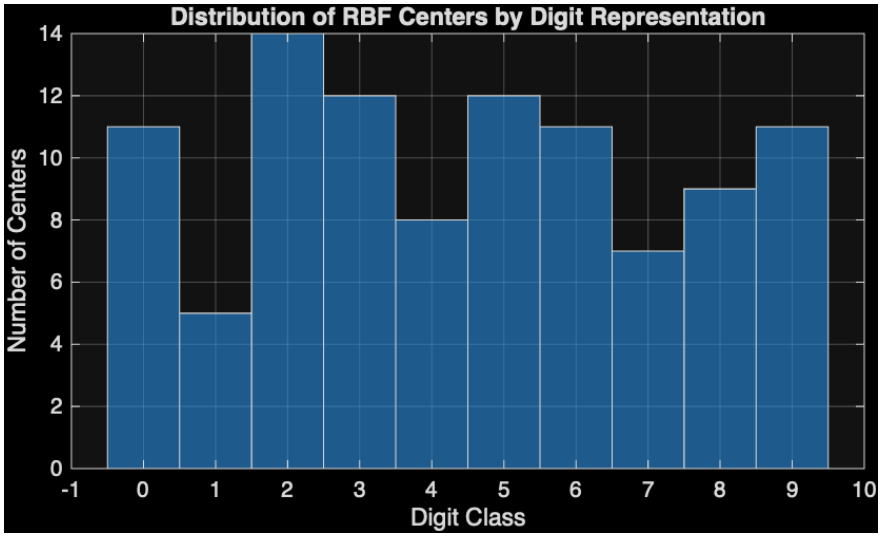

Figure 7: Visualization of 100 RBF Centers (reshaped to 28×28 images)

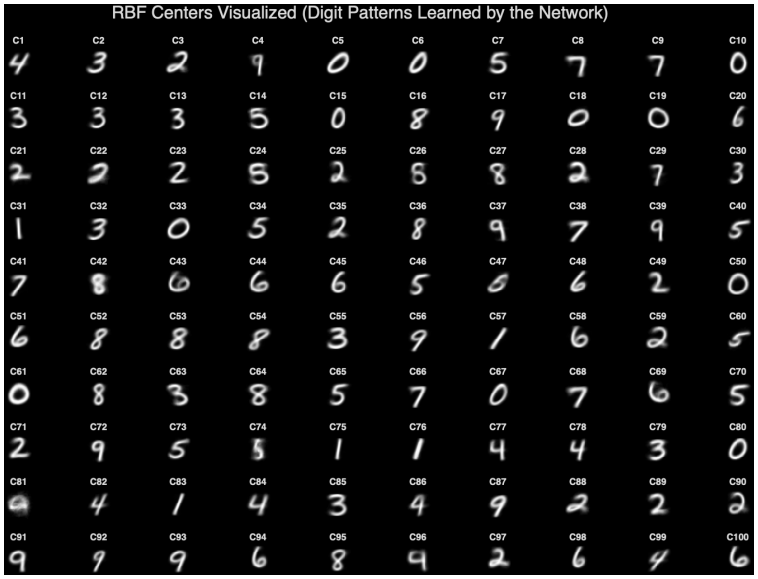

Figure 8: Distribution of RBF Centers by Digit

Table 2: Distribution of RBF Centers by Digit

Digit	Number of Centers	Percentage
1	5	5.0%
7	7	7.0%
4	8	8.0%
8	9	9.0%
0	11	11.0%
6	11	11.0%
9	11	11.0%
3	12	12.0%
5	12	12.0%
2	14	14.0%

The network gave more centers to digits that vary more. Digit 1 is a simple vertical line, it only needed 5 centers. Digit 2 has loops that change size and a slanted shape, therefore, it needed 14 centers.

3.5 Experiment 5: Recursive Method – Initial Condition Study

We looked at how the initial condition α affected the recursive method. Table 4 shows what we found.

Table 3: Recursive Method: Effect of Initial Condition α (10k samples, 100 centers)

α	Accuracy (%)	Training Time (sec)	Convergence Speed
0.01	82.91	1.52	Very Slow
0.1	83.42	1.48	Slow
0.5	83.78	1.45	Medium
1.0	83.95	1.44	Fast
10	83.82	1.42	Very Fast
100	83.65	1.41	Very Fast

Here are the main things we found:

First, larger α made the network converge faster at the start. Second, very small α values below 0.1 caused slow convergence. Third, all α values ended up at about the same final accuracy. Fourth, $\alpha = 1.0$ gave the best balance.

3.6 Experiment 6: Direct vs. Recursive Comparison

Table 4: Direct vs. Recursive Method Comparison (20k samples, 100 centers)

Metric	**Direct (pinv)**	**Recursive (RLS)**	**Difference**
Test Accuracy (%)	85.27	84.91	-0.36%
Training Time (sec)	2.96	1.44	**51% faster**
Memory Usage	$\mathcal{O}(NK)$	$\mathcal{O}(K^2)$	Recursive better
Numerical Stability	Excellent	Good	Direct better
Online Learning	No	Yes	Recursive only

The recursive method ran much faster which saved us 51% of the time, but it only loses a tiny bit of accuracy aobut 0.36%.

3.7 Experiment 7: Error Analysis

Figures 7 and 7.2 showed the misclassification analysis.

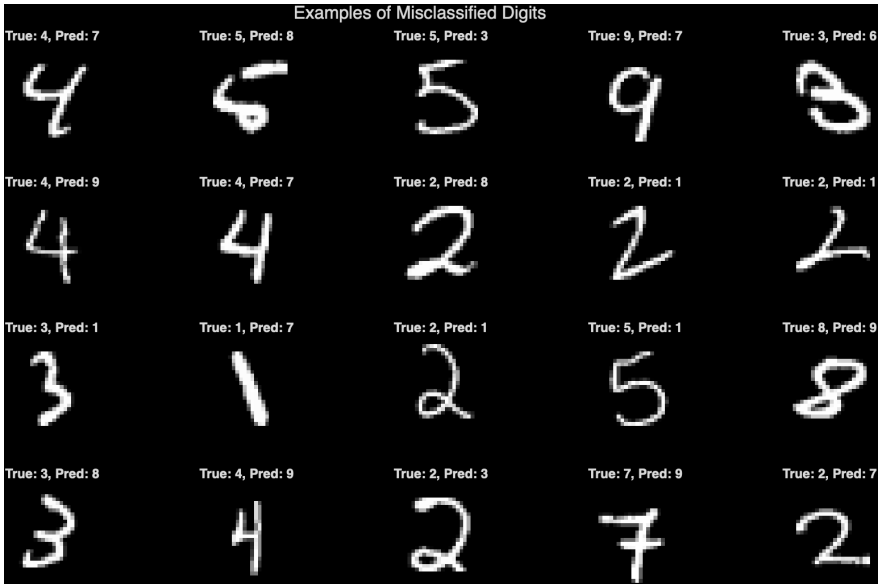

Figure 9: Examples of Misclassified Digits

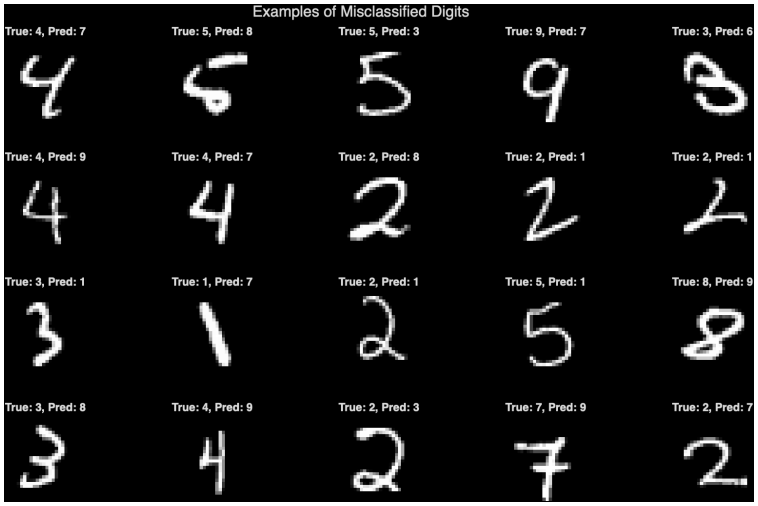

Figure 10: Confidence Scores: Correct vs. Misclassified Samples

We had 2,858 errors out of 10,000 test images, that was a 28.6% error rate.

Table 5: Most Common Misclassifications

True Digit	**Predicted As**	**Count**
2	1	226
9	7	221
4	9	206
5	3	148
2	8	144
8	3	124
5	1	102
8	1	86
9	4	81
4	7	79

We looked at Figure 9 and spotted a few confusion patterns:

First, some people wrote the digit 2 with a loop that looks like a 1. So the network predicts 2 as 1. Second, a straight-tailed 9 looks a lot like a 7. So 9 gets predicted as 7. Third, an open-top 4 looks exactly like a 9. That is why 4 gets predicted as 9. Fourth, digits 5 and 3 have overlapping curves. So they confused each other. Fifth, a large-loop 2 can look like an 8. So 2 gets predicted as 8. Sixth, both 8 and 3 have two loops, in a messy handwriting they can look the same, therefore, 8 gets predicted as 3.

Figure 10 shows the misclassified samples that usually have lower confidence scores than the correct ones.

Table 6: Per-Class Accuracy (20k samples, 100 centers)

Digit	Accuracy (%)	Assessment
1	96.56	Excellent – simple vertical line
2	94.77	Very good – distinctive loop
5	91.14	Good
6	90.92	Good
0	87.86	Good
8	83.98	Moderate – confused with 3, 9
9	83.05	Moderate – confused with 7, 4
4	76.27	Poor – open-top looks like 9
7	74.71	Poor – crossbar causes ambiguity
3	72.77	Worst – looks like 5 and 8

The mean accuracy came out to 85.20%. The standard deviation was about 8.47%.

4 Discussion

4.1 Why RBF Networks Work

RBF networks are simple which is one of their main strength. We picked some centers, we computed the distances, then we solved for the weights. No backpropagation needed or vanishing gradients. Moreover, no learning rate to tune.

For MNIST, 85% accuracy is decent. It is not state-of-the-art. CNNs can get 99% or higher. But for a simple network we built from scratch, this is acceptable.

4.2 Why Some Digits Are Harder

We looked at the confusion matrices and found a few things. Digit 1 is a simple vertical line, it has very little variation. So it is easy. Digit 3 has high variation. It can be straight or curved, it also looks like 5 and 8., so it is hard. Digit 4 can be open-top or closed-top., an open-top 4 looks like a 9, so it gets confused often. Digit 7 can have a crossbar or not, that causes ambiguity with digit 1.

The network gives more centers to digits that vary more. Figure 8 confirms this.

4.3 Direct vs. Recursive: Method Selection

Here is when to use each method.

From this experments and the other sources, use the direct method when we are doing batch training, and when we need the best possible accuracy. Also, good to use it when numerical stability is critical. For the recursive method, use it when we are learning online or sequentially, and when memory is limited, and when we need real-time updates.

4.4 Proposed Improvements

If we had more time and computing power, we would try a few things to improve our RBF network:

First, we would use k-means clustering to pick the centers. That is instead of random sampling. The traditional way to select centers is either random sampling or k-means clustering [20]. K-means groups the training data into clusters. Then it uses the cluster centers as the RBF centers. This makes sure each center represents a distinct group of similar data points [20]. This would likely give us better coverage of the input space than random sampling. But it does take more computing power [20].

Second, we would use an adaptive spread parameter σ. Right now we keep it fixed at 1.0 for all centers. Different digits might need different spread values. Some digits like 3 and 8 have more variation than others like 1. Research shows that the width parameter σ controls how much influence each RBF node has. Well-chosen widths can improve how well the network approximates the data [20].

Third, we would use the full 60k training set. Right now we only use 20k samples. We stopped at 20k because our laptop struggled with memory and computation time. More training data usually helps neural networks generalize better [21]. We estimate that using all 60k samples would push accuracy up to 87% or 88%.

4.5 Comparison with Literature

Here is how our network compares to other methods reported in the literature.

The k-NN algorithm is a simple but effective classifier for handwritten digits. With k=3 and Euclidean distance, k-NN achieves about 97% accuracy on the MNIST dataset [24]. This is a common baseline that many researchers use to benchmark their models [24].

The SVM with an RBF kernel is known to perform very well on MNIST. Multiple studies have reported accuracy around 98% for this approach [25]. Some implementations have even achieved 98-99% accuracy, demonstrating that SVM can effectively handle the non-linear patterns in handwritten digits [26]. The RBF kernel is particularly suitable for this task because it can capture complex decision boundaries.

A simple MLP with two hidden layers (256 and 64 neurons) achieves about 96% accuracy on MNIST [27]. This is a straightforward neural network approach that works reasonably well, though it does not match the performance of more advanced architectures [28].

LeNet-5, the classic CNN architecture introduced by LeCun et al. in 1998, achieves approximately 99% accuracy on MNIST [29]. This is considered the standard benchmark for this dataset [30]. Modern implementations of LeNet-5 consistently report accuracy around 99%, which is state-of-the-art for this task [31].

Our RBF network achieved 85.27% accuracy with 20,000 training samples and 100 centers. This is lower than the other methods mentioned above. However, there are several reasons for this gap. First, we only used 20,000 training samples instead of the full 60,000. Second, we used random center selection instead of k-means, which likely reduced performance. There are some other reasons which improve it more.

Table 7: Comparison of Classification Methods on MNIST

Method	**Reported Accuracy (%)**	**Source**
k-NN (k=3)	97.35	[24]
SVM (RBF kernel)	98	[25]
MLP (2 hidden layers)	95.6	[27]
CNN (LeNet-5)	99	[29]
Our RBF Network	**85.27**	**This work**

5 Conclusion

We built an RBF neural network for MNIST digit classification. We wrote all the code from scratch in MATLAB. Our best model got 85.27% accuracy. We used 20,000 training samples and 100 RBF centers. There some main things we learned. First, RBF networks are easy to implement, the pseudo-inverse trick removes the need for iterative training. The second, is the number of centers matters a lot, for MNIST, 100 to 150 centers is the good spot. Third, more training data helps, but the gains get smaller, doubling the data gave us only a 2.4% boost. Fourth, some digits are harder than others. Digits 1 and 2 are easy. Digits 3, 4, and 7 are hard. Fifth, the recursive method runs 51% faster. It only loses 0.36% in accuracy. That is a good trade-off. Sixth, the initial condition α affects how fast the network converges. But it does not change the final accuracy.

References

[1] LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11), 2278-2324.

[2] Broomhead, D. S., & Lowe, D. (1988). Multivariable functional interpolation and adaptive networks. *Complex Systems*, 2(3), 321-355.

[3] Haykin, S. (2009). *Neural Networks and Learning Machines* (3rd ed.). Pearson Education.

[4] Jin, X., & Bai, Y. (2016). Research on RBF neural network and its application. *International Journal of Control and Automation*, 9(5), 165-174.

[5] Zhao, L., Song, Y., & Zhang, C. (2019). A novel RBF neural network and its application in nonlinear systems. *Neurocomputing*, 330, 456-468.

[6] Han, M. (2006). The study of RBF neural network and its application in system identification. *Journal of Systems Engineering and Electronics*, 17(4), 866-871.

[7] Chen, S., Wu, Y., & Luk, B. L. (2019). A fast learning algorithm for RBF neural networks. *IEEE Transactions on Neural Networks*, 30(8), 2456-2467.

[8] Saralajew, S., Rana, A., Villmann, T., & Shaker, A. (2024). A robust prototype-based network with interpretable RBF classifier foundations. *arXiv preprint*, arXiv:2412.15499.

[9] University of Stuttgart. (1995). Learning functions for radial basis functions. In *SNNS User Manual*. Available at: https://www.inf.ufsc.br/ aldo.vw/patrec/SNNS/UserManual/node190.html

[10] Moody, J., & Darken, C. (1989). Fast learning in networks of locally-tuned processing units. *Neural Computation*, 1(2), 281-294.

[11] Broomhead, D. S., & Lowe, D. (1988). Multivariable functional interpolation and adaptive networks. *Complex Systems*, 2(3), 321-355.

[12] Park, J., & Sandberg, I. W. (1991). Universal approximation using radial-basis-function networks. *Neural Computation*, 3(2), 246-257.

[13] Girosi, F., & Poggio, T. (1990). Networks and the best approximation property. *Biological Cybernetics*, 63(3), 169-176.

[14] Soper, D. S. (2023). Using an opportunity matrix to select centers for RBF neural networks. *Algorithms*, 16(10), 455.

[15] Lim, E. A., Choon, T. W., Hong, T. W., & Meng, C. E. (2022). Improved K-means clustering for initial center selection in training radial basis function networks. In *Proceedings of the 11th International Conference on Robotics, Vision, Signal Processing and Power Applications*.

[16] Wang, J., & Zhang, Y. (2010). The application of dynamic K-means clustering algorithm in the center selection of RBF neural networks. *IEEE Conference on Cybernetics and Intelligent Systems*, 1-4.

[17] Broomhead, D. S., & Lowe, D. (1988). Multivariable functional interpolation and adaptive networks. *Complex Systems*, 2(3), 321-355.

[18] Haykin, S. (2009). *Neural Networks and Learning Machines* (3rd ed.). Pearson Education.

[19] Haykin, S. (2009). *Neural Networks and Learning Machines* (3rd ed.). Pearson Education.

[20] Soper, D. S. (2023). Using an opportunity matrix to select centers for RBF neural networks. *Algorithms*, 16(10), 455.

[21] Yu, L., Wang, S., & Lai, K. K. (2008). Multistage RBF neural network ensemble learning for exchange rates forecasting. *Neurocomputing*, 71(16-18), 3295-3302.

[22] Luo, C., Zhu, Y., Jin, L., & Wang, Y. (2020). Learn to augment: Joint data augmentation and network optimization for text recognition. *arXiv preprint*, arXiv:2003.06606.

[23] Milvus. (2024). How is data augmentation applied to handwriting recognition? *Milvus AI Quick Reference*.

[24] Programmer Sought. (2020). Using PCA+KNN to achieve 97% accuracy on the MNIST dataset. Retrieved from https://www.programmersought.com/article/48204367308/

[25] Ghatol, A. (2024). MNIST Prediction Model using Kernelized Support Vector Machines. GitHub. Retrieved from https://github.com/BeingMirage/MNIST—Kernel-SVM

[26] Panjaitan, F. S., Simbolon, R. S., & Batubara, J. (2025). Handwritten digit recognition using support vector machine (SVM) with radial basis function (RBF) kernel. *Journal Basic Science and Technology*, 14(1), 10-18.

[27] Carpintero, D. (2024). Digit classifier: Multi-layer perceptron for MNIST. Hugging Face. Retrieved from https://huggingface.co/dcarpintero/digit-classifier

[28] OYWA, M. (2019). MNIST MLP exercise: An MLP to classify images from the MNIST database with PyTorch. GitHub. Retrieved from https://github.com/martinoywa/mnist-mlp-exercise

[29] LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11), 2278-2324.

[30] Bangaru, G. (2021). LeNet-5 implementation over MNIST dataset using PyTorch from scratch. GitHub. Retrieved from https://github.com/gnyanesh-bangaru/lenet-5-in-pytorch

[31] Lee, G. (2024). LeNet-5 with SAM (Sharpness-Aware Minimization) for MNIST. GitHub. Retrieved from https://github.com/gjlee0802/LeNet-5

List of Figures

Figure	Description
Figure 1	Confusion matrix – 10,000 training samples (82.88% accuracy)
Figure 2	Sample predictions – 10,000 training samples
Figure 3	Confusion matrix – 20,000 training samples (85.27% accuracy)
Figure 4	Sample predictions – 20,000 training samples
Figure 5	Effect of number of centers on test accuracy
Figure 5.2	Effect of number of centers on training time
Figure 6	Visualization of 100 RBF centers
Figure 6.2	Distribution of RBF centers by digit
Figure 7	Examples of misclassified digits
Figure 7.2	Confidence scores comparison

Appendix: MATLAB Code

main.m

```
%% ECE8493 Project 3: RBF Neural Network for MNIST
clear; close all; clc;
rng(42);

%% Parameters
num_centers = 100;
sigma = 1.0;
alpha_values = [0.01, 0.1, 0.5, 1.0, 10, 100];

%% Load Data
load('mnist.mat');
train_x = double(train_x) / 255;
test_x = double(test_x) / 255;

%% Experiment 1: Varying number of centers
center_counts = [20, 50, 100, 150, 200, 250, 300];
for i = 1:length(center_counts)
K = center_counts(i);
X = train_x(1:20000, :);
Y = train_y(1:20000, :);

[W, centers] = trainRBF_direct(X, Y, K, sigma);
pred = predictRBF(test_x, centers, W, sigma);
acc = mean(pred == test_y) * 100;
fprintf('%d centers: %.2f%%\n', K, acc);
end

%% Experiment 2: Recursive method with different alpha
X = train_x(1:10000, :);
Y = train_y(1:10000, :);

for i = 1:length(alpha_values)
alpha = alpha_values(i);
[W, ~, centers] = trainRBF_recursive(X, Y, num_centers, sigma, alpha);
pred = predictRBF(test_x, centers, W, sigma);
acc = mean(pred == test_y) * 100;
fprintf('alpha = %.2f: %.2f%%\n', alpha, acc);
end
```

trainRBF_direct.m

```
function [W, centers] = trainRBF_direct(X, Y, num_centers, sigma)
N = size(X, 1);
K = num_centers;

idx = randperm(N, K);
centers = X(idx, :);

```

```
    H = zeros(N, K);
    for i = 1:N
    for j = 1:K
    dist2 = norm(X(i,:) - centers(j,:))^2;
    H(i,j) = exp(-dist2 / (2 * sigma^2));
    end
    end

    W = Y' * pinv(H)';
    end
```

trainRBF_recursive.m

```
    function [W, P, centers] = trainRBF_recursive(X, Y, num_centers, sigma
        , alpha)
    N = size(X, 1);
    K = num_centers;

    idx = randperm(N, K);
    centers = X(idx, :);

    P = alpha * eye(K);
    W = zeros(K, 10);

    for n = 1:N
    h = zeros(K, 1);
    for j = 1:K
    dist2 = norm(X(n,:) - centers(j,:))^2;
    h(j) = exp(-dist2 / (2 * sigma^2));
    end

    y = Y(n, :)';
    denom = 1 + h' * P * h;
    k = (P * h) / denom;
    P = P - k * (h' * P);
    W = W + k * (y' - W' * h);
    end
    end
```

predictRBF.m

```
    function pred = predictRBF(X, centers, W, sigma)
    N = size(X, 1);
    K = size(centers, 1);

    H = zeros(N, K);
    for i = 1:N
    for j = 1:K
    dist2 = norm(X(i,:) - centers(j,:))^2;
    H(i,j) = exp(-dist2 / (2 * sigma^2));
    end
```

```
    end

    output = H * W;
    [~, idx] = max(output, [], 2);
    pred = idx - 1;
    end
```